Project Group 27 Proposal: Comparing GA, PSO, and ACO for Pathfinding in 2D Mazes

Manon Leijser Nemo Ingendaa Robert Blaauwendraad

June 2, 2026

1 Problem Statement

While standard algorithms like A* excel in static environments and simple grids, they often struggle in unpredictable environments with deceptive dead ends. When an environment suddenly changes, these solvers often have to start over. In contrast, natural computing algorithms use a population to solve problems, allowing the group to adapt together.

The central theoretical question of this project is not which algorithm performs best, but how different information architectures shape a population's adaptive behavior in environments that impose different challenges. We focus on three paradigms representing distinct architectures of inter-agent coupling:

- **Genetic Algorithms (GA)** are decoupled. Agents share no direct information. Knowledge is passed between generations through selection pressure and recombination.
- **Particle Swarm Optimization (PSO)** is tightly coupled. Every agent immediately knows the group's best position, producing high social pressure.
- **Ant Colony Optimization (ACO)** is environmentally coupled. Agents communicate indirectly through pheromone trails on the grid. The environment itself becomes the collective memory.

These paradigms give us a natural axis along which to study coupling: no shared memory (GA), continuous global memory (PSO), and distributed environmental memory (ACO). By evaluating them in environments designed to stress different coupling mechanisms, we can characterize when each architecture fails and how effectively a population recovers its diversity.

Research Question. To what extent do the information architectures of GA, PSO, and ACO, defined by their varying degrees of agent coupling, shape a population's adaptive behavior when the environment imposes distinct challenges of deception, non-stationarity, and multimodality?

We decompose this into three subquestions:

- **SQ1 (architecture).** How does the coupling mechanism of each algorithm determine the way its population distributes itself across the search space over time?
- **SQ2 (environment).** How does the type of environmental challenge interact with each coupling architecture to produce distinct failure and recovery patterns?
- **SQ3 (scale).** To what extent does population size modulate adaptive capacity, and does increasing scale compensate for or amplify architectural weaknesses?

2 Hypotheses

We organize our predictions into three hypothesis families, each tied to a subquestion.

H1 (Coupling shapes diversity dynamics; SQ1). Each architecture produces a distinct entropy signature when agents explore the search space. We expect PSO to show the sharpest entropy collapse on encountering a strong attractor, since the global best pulls all particles simultaneously toward the same position. We expect GA to maintain more stable entropy, because decoupled agents cannot lock onto a single misleading path together. We expect ACO to sit between the two, with gradual rather than catastrophic decline, because pheromone trails build and erode over time rather than updating instantaneously.

H2 (Environmental challenge interacts with coupling; SQ2). Each maze type should stress a different architecture most strongly. Deception (Maze 1) should affect PSO most, because tight coupling accelerates commitment to the misleading attractor. Non-stationarity (Maze 2) should affect ACO most, because strong pheromone trails on invalidated paths persist until evaporation enables rerouting. Multimodality (Maze 3) should affect PSO most in terms of spatial spread, because social pressure forces the swarm onto a single path even when alternatives are equally valid.

H3 (Population scale modulates but does not reverse architectural effects; SQ3). We expect larger populations to amplify PSO's weakness rather than compensate for it. More particles pulled toward the same wrong attractor still produces collapse, only with more agents involved. We expect GA to benefit most from scale, since decoupled stochastic variation improves with additional independent samples. We expect ACO to be comparatively insensitive to population within the tested range, because pheromone dynamics dominate over ant count.

3 Related Work

Sariff and Buniyamin [6] applied GA and ACO to robot path planning in static environments, finding that ACO becomes more robust as complexity increases. Shrestha et al. [5] extended this line by benchmarking GA, PSO, and ACO on a common 2D environment with parameters tuned for a waypoint-based representation.

To characterize population-internal dynamics, we track spatial diversity using Shannon entropy. Masisi et al. [4] demonstrated that higher diversity consistently produces better outcomes in optimization tasks. We adapt this measure to the spatial distribution of agents across the maze, giving us a quantitative signal for premature convergence and recovery speed rather than just final performance.

Ansari et al. [2] studied efficiency in complex pathfinding, noting that heuristic searches require excessive iterations to escape dead ends. We use their efficiency framework to structure our benchmarks, while extending the approach to investigate the underlying causes of these delays.

Our work differs from Shrestha et al. [5] in four ways. First, we introduce dynamic environments that test adaptation under disruption, which Shrestha et al. identify as future work in their conclusion. Second, we measure population-internal dynamics via Shannon entropy rather than only endpoint metrics such as convergence time and path length. Third, we isolate specific environmental properties, namely deception, non-stationarity, and multimodality, rather than varying complexity monotonically through obstacle count. Fourth, we frame the three algorithms as instances of distinct information architectures and derive predictions about failure modes from that framing, rather than treating them as interchangeable metaheuristics. Together these

extensions fill a gap in the comparative literature: how memory architectures degrade and recover when the environment itself changes.

4 Approach

The simulation framework is built from scratch in Python, utilizing code from previous assignments for the core GA and PSO implementations.

4.1 Environment and Algorithm Setup

We model the environment as a 2D grid where agents must navigate from a start node to a goal node in the fewest steps.

Algorithm parameters are drawn from multiple literature sources, chosen to match each algorithm's most comparable problem formulation. No single paper tunes all three algorithms on a grid-based maze with cell-by-cell navigation, so we combine sources and document each choice.

GA parameters Population 50, roulette wheel selection, two-point crossover at common cells with rate 0.5, per-chromosome mutation at rate 0.3 via truncate-and-regrow. Chromosomes encode a variable-length sequence of cells visited from start, with the cell-sequence representation following Lamini et al. [3] and the variable-length grid chromosome following Tu & Yang [7]. This differs from Shrestha et al. [5], whose fixed-width chromosome encodes indices into a set of randomly scattered waypoints in a 2D Euclidean workspace, with path segments running through obstacle-free space between waypoints. In our grid-based maze formulation the robot moves between adjacent cells, so the path itself is the chromosome's natural representation and no inter-cell interpolation step exists to be encoded. The numerical parameters and operator types follow Shrestha's specification, with adaptations needed to fit the cell-sequence chromosome. Roulette wheel selection and the mutation rate of 0.3 transfer without change, and the population size of 50 matches Tu & Yang's setup. Two-point crossover is restricted to cells shared between parents, an adaptation in the broader family of common-cell and same-adjacency operators developed for variable-length path chromosomes. Mutation shifts from per-bit flip to per-chromosome truncate-and-regrow, since per-cell flipping would break adjacency and produce invalid paths.

PSO parameters (Shrestha et al. [5]). Swarm size 50, inertia $\omega = 1.0$, cognitive coefficient $c_1 = 0.1$, social coefficient $c_2 = 0.2$. The random coefficients r_1 and r_2 in the velocity update are runtime draws from $U(0, 1)$ rather than hyperparameters.

ACO parameters (Akka and Khaber [1] for structural parameters; Shrestha et al. [5] for evaporation and initial pheromone). Ant colony size 50, pheromone exponent $\alpha = 1$, heuristic exponent $\beta = 5$, pheromone deposit $Q = 2$, evaporation rate $\rho = 0.1$, initial pheromone 0.8. Akka and Khaber use a grid-based ACO formulation with cell-by-cell navigation that matches our setup more closely than either Sariff and Buniyamin [6], which operates on feasible-node graphs, or Shrestha et al. [5], which uses waypoints. We scale the ant count from their reported 30 to 50 to align with the unified population used across all three algorithms.

We unify population at 50 across all three algorithms, which differs from Shrestha's per-algorithm values of 70, 40, and 20. Comparing architectures at a shared scale isolates coupling effects from population effects, which is what our research question requires. Population variation is a deliberate axis of our experimental design rather than a confound.

4.2 Diversity Methodology

A central tension in natural computing is the balance between exploration and exploitation. We operationalize this as spatial diversity: the degree to which agents are distributed across structurally distinct regions of the maze at any given iteration.

We quantify this using position-wise Shannon entropy:

$$H_i = -\sum_{j \in \Sigma} f_{ij} \log f_{ij}$$

where f_{ij} is the frequency of agents occupying cell j at iteration i. A high value indicates active multi-path exploration; a sudden drop signals convergence.

4.3 Environments

We use three maze types, each designed to isolate a specific environmental challenge. For each type we generate 10 distinct instances that preserve the structural property of that maze type, and we run 10 independent trials per instance. This yields 100 runs per condition, and separates algorithmic stochasticity from maze-instance variance.

Maze 1: U-Shaped Trap (deception). Features a U-shaped dead end placed near the goal. Agents are initially attracted toward the goal and enter the trap. Escaping requires moving away from the goal, against the heuristic gradient. This tests whether PSO becomes trapped when all particles follow a misleading global best, whether ACO wastes iterations strengthening useless pheromone trails in the dead end, and whether GA's stochastic mutation allows escape.

Maze 2: Sudden Wall (non-stationarity). Two routes exist: a short path and a longer alternative. The short path is blocked at iteration $T = 100$, after the population has settled into it. This tests recovery from environmental change. We expect PSO to pile up at the new wall because particles follow the leader now stuck there, ACO to persist with strong outdated pheromone trails until evaporation, and GA to adapt most quickly through stochastic variation.

Maze 3: Parallel Paths (multimodality). Two routes of identical length are available at the start, with a small detour near the end of one route. This tests whether algorithms can spread across multiple valid paths simultaneously. We expect PSO to collapse onto one side due to social pressure, GA to struggle when crossover combines agents from opposite paths, and ACO to explore both routes and eventually favor the slightly shorter one.

4.4 Experimental Design

We run a single consolidated experiment with three factors: algorithm, maze type, and population size. This design answers all three subquestions from one coherent data set rather than treating them as separate studies.

Algorithms: GA, PSO, ACO (addresses SQ1). Maze types: U-Shaped Trap, Sudden Wall, Parallel Paths (addresses SQ2). Population sizes: 20, 50, 150 (addresses SQ3). The range covers Shrestha's reported values from 20 to 70 and extends to 150 to test scale effects.

Each cell of the $3 \times 3 \times 3$ factorial is run on 10 distinct maze instances with 10 independent trials per instance. Total computational budget: $3 \times 3 \times 3 \times 10 \times 10 = 2{,}700$ runs. Based on preliminary timing this is feasible on a single machine within the project timeline.

The three-way factorial lets us analyze main effects (does coupling matter, does environment matter, does scale matter) and interactions (do population effects depend on environment, do

coupling effects depend on scale). Interaction effects are where the more interesting findings typically live and are not accessible from a purely additive design.

4.5 Evaluation Metrics

We distinguish primary metrics, which directly address the subquestions, from context metrics used for grounding.

Shannon entropy over iterations (addresses SQ1). The shape of the entropy curve is our main diagnostic of coupling behavior. A sharp cliff indicates rapid collapse consistent with tight coupling. A slow drift indicates lag in environmental memory. A noisy plateau indicates stochastic wandering. For statistical comparison we derive three scalars. Time to 50% entropy loss measures the speed of collapse. Diversity floor, defined as the minimum entropy reached during a trap or disruption, measures the severity of collapse. Mean entropy across the run measures sustained diversity.

Adaptation time in Maze 2 (addresses SQ2). Iterations from disruption at $T = 100$ until the population's mean entropy first returns to 80% of its pre-disruption value. A threshold is required because entropy recovers gradually and "recovered" has no natural cutoff. We apply the same 80% threshold to every algorithm, population size, and maze instance.

Success rate across 100 runs per cell. Defined as reaching the goal within the iteration limit. High diversity means nothing if the algorithm never finds the goal. Success rate grounds our entropy analysis in actual performance and flags any case where coupling effects prevent task completion.

Path optimality relative to the A* optimum. Computed for successful runs only. Ensures high-diversity behavior is not just noise, and provides a deterministic anchor for our results.

4.6 Limitations

We fix algorithm-specific parameters using values from the literature and do not perform parameter tuning. Our findings about coupling architectures are therefore conditional on these parameter choices. This is particularly relevant for ACO's evaporation rate ρ, which directly controls the persistence of pheromone trails. Our choice of $\rho = 0.1$ produces long pheromone memory, which amplifies the adaptation effects predicted by H1c and H2b. Higher ρ would weaken both predicted effects. We acknowledge this dependency openly rather than claim robustness we have not tested.

Shrestha et al. [5] use a waypoint-based path representation, whereas our grid-based formulation with cell-by-cell navigation and wall collisions differs in its dynamics. Parameters adapted from their study therefore serve as a reasonable starting point but cannot be assumed optimal for our setup. Akka and Khaber [1] use a grid-based formulation matching ours, which is why we adopt their structural ACO parameters.

The GA case sharpens this point empirically. We initially implemented a binary direction-string encoding, with each chromosome a fixed-length sequence of U/D/L/R moves of length $2(W + H)$, chosen so that Shrestha's bit-flip mutation at rate 0.3 could be applied to the underlying bits. This produced 0% success across all nine experimental cells covering three mazes and three population sizes, at 100 trials per cell. Two diagnostic sweeps confirmed the failure was structural rather than parametric. A 4×2 sweep over mutation rate and chromosome length at 40 trials gave 0% success across all cells. An iteration-budget sweep up to 50,000 generations at population 50, equivalent to 2.5M chromosome evaluations per trial, gave 0%

success across 16 trials and every budget, with mean best-found distance to goal plateauing at roughly 23 cells regardless of compute.

Direction-string crossover has no inheritable invariant. Splicing two strings that happen to reach the goal almost never produces another goal-reaching string, because reachability depends on the move sequence as a whole rather than on any local property of the genes. Cell-sequence encodings are robust to this because two-point crossover at cells shared between parents preserves connectivity by construction, so any spliced child is a structurally valid path. Direction encoding can work in other settings, as Tu & Yang [7] demonstrate in small grids with a different fitness landscape and a distance bit per gene, but in our maze setting the encoding's fragility prevented convergence at any compute budget we tested. The lesson is that operator parameters cannot be ported across encodings without checking whether the encoding's invariants survive recombination.

Neither Shrestha et al. [5], Sariff and Buniyamin [6], nor Akka and Khaber [1] evaluate dynamic environments. Our Maze 2 findings therefore extend the literature into territory where parameter choices have not been validated.

5 Planning

Week (Start)	**Milestones, Work Periods, and Deadlines**
Week 1 (Apr 06)	Finalize Proposal. Checkpoint 0: Apr 10.
Week 2 (Apr 13)	Environment setup and maze instance generation.
Week 3 (Apr 20)	Complete GA prototype to satisfy Checkpoint 1. Begin PSO and ACO implementation. Draft Report: Intro and Related Work.
Week 4 (Apr 27)	Finalize full implementation of PSO and ACO. Draft Report: Methodology and Setup.
Week 5 (May 04)	Run Experiment 1 across all three mazes at fixed population.
Week 6 (May 11)	Run Experiments 2 and 3. Synthesize initial data. Draft Report: Results on static mazes. Checkpoint 2: May 13.
Week 7 (May 18)	Design presentation. Draft Report: Results on dynamic maze and sensitivity. Checkpoint 3: May 24.
Week 8 (May 25)	Flash Talks: May 27. Draft Report: Discussion and Conclusion. Checkpoint 4: May 28.
Week 9 (Jun 01)	Compile full report, conduct final analysis, and revise based on supervisor feedback.
Week 10 (Jun 08)	Final proofreading and formatting. Final Project Report Submission: Jun 12.

References

[1] Khaled Akka and Farid Khaber. Mobile robot path planning using an improved ant colony optimization. *International Journal of Advanced Robotic Systems*, 15:172988141877467, 05 2018.

[2] Ahlam Ansari, Mohd Amin Sayyed, Khatija Ratlamwala, and Parvin Shaikh. An optimized hybrid approach for path finding, 2015.

[3] Said Benhlima, Lamini Chaymaa, and Ali Bekri. Genetic algorithm based approach for autonomous mobile robot path planning. *Procedia Computer Science*, 127, 03 2018.

[4] L. Masisi, V. Nelwamondo, and T. Marwala. The use of entropy to measure structural diversity, 2008.

[5] Praveen Reddy Kota Alaa Sheta Santosh Shrestha, Pranith Varma Appani. Optimizing robot path planning in 2d static environments using ga, pso and aco search algorithms. *International Journal of Computer Applications*, 186(7):1–10, Feb 2024.

[6] Nohaidda Binti Sariff and Norlida Buniyamin. Comparative study of genetic algorithm and ant colony optimization algorithm performances for robot path planning in global static environments of different complexities, 2009.

[7] Jianping Tu and S.X. Yang. Genetic algorithm based path planning for a mobile robot. In *2003 IEEE International Conference on Robotics and Automation (Cat. No.03CH37422)*, volume 1, pages 1221–1226 vol.1, 2003.